Reviewer Pack — Submodular Measure on Monotone DNF for k-CLIQUE Approximatio…

Entry c4e28a8fc7d7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 05:13:43 UTC

Submodular Measure on Monotone DNF for k-CLIQUE Approximation

Entry ID: `c4e28a8fc7d7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 05:13:43 UTC

1. Conjecture

Field A (mathematical branch): MATROID_THEORY **Field B** (complexity object): BOOLEAN_FUNCTION_APPR

Statement:

> There exists a submodular measure μ on monotone DNF formulas such that μ(f) ≤ O(log n) for any DNF formula f of size poly(n), but μ(k-CLIQUE) ≥ Ω(n) for all k ≥ 2. The measure μ is defined as the minimal number of terms in a DNF that approximates the k-CLIQUE function with error ≤ $1/n^2$ under uniform distribution.

Rationale (proposer's reasoning):

> This conjecture leverages matroid rank functions (submodularity) to quantify the structural complexity of DNF approximations. By tying μ to the approximation error of k-CLIQUE, it bridges matroid theory (field_A) with boolean function approximation (field_B). If true, it would provide a new combinatorial barrier for monotone circuits, as the submodular measure would grow linearly with n for k-CLIQUE but remain logarithmic for other functions.

Taxonomy category: `PROOF_COMPLEXITY_TSEITIN` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `67b22fc81f538d3a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `submodular measure monotone DNF k-CLIQUE approximation` - `matroid theory boolean function approximation submodular measure DNF` - `k-CLIQUE approximation DNF submodular measure error`

Top relevant hits considered: - [s2:1512.03726] Quantitative estimates in approximation by Bernstein-Durrmeyer-Choquet operators with respect to monotone and submodular - [s2:2308.05588] Banzhaf Values for Facts in Query Answering

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=4.6s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_prime(n):
        if n <= 1:
            return False
        for i in range(2, int(math.sqrt(n)) + 1):
            if n % i == 0:
                return False
        return True

    primes = [i for i in range(5, 40) if is_prime(i)]

    def generate_dnf(n):
        terms = []
        for _ in range(random.randint(2, 5)):
            term = set()
            for j in range(n):
                if random.random() < 0.3:
                    term.add(j)
            terms.append(term)
        return terms

    def compute_mu(dnf, k):
        n = len(dnf[0])
        target_cliques = []
        for i in range(1 << n):
```

```
            clique = set()
            for j in range(n):
                if i & (1 << j):
                    clique.add(j)
            if len(clique) == k and all(term.issubset(clique) for term in dnf):
                target_cliques.append(clique)

        min_terms = float('inf')
        for _ in range(100):  # Randomized search
            selected_terms = random.sample(dnf, len(dnf))
            covered = set()
            for term in selected_terms:
                covered.update(term)
            if all(len(covered & clique) > 0 for clique in target_cliques):
                min_terms = min(min_terms, len(selected_terms))
        return min_terms

    def k_clique(n, k):
        edges = []
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.append((i, j))
        return edges

    results = []
    for n in primes:
        dnf = generate_dnf(n)
        mu = compute_mu(dnf, k=2)  # Example for k=2
        results.append({"n": n, "mu": mu})

    mean_mu = sum(result["mu"] for result in results) / len(results)
    std_mu = math.sqrt(sum((result["mu"] - mean_mu) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["mu"] >= n) / len(results)

    return {
        "metric_name": "submodular_measure",
        "metric_value": mean_mu,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction >= 0.8,
        "counterexample": "" if support_fraction >= 0.8 else f"n={results[0]['n']}, mu={results[0]
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or primes
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_mu = sum(result["metric_value"] for result in results) / len(results)
    std_mu = math.sqrt(sum((result["metric_value"] - mean_mu) ** 2 for result in results) / len(re
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
```

```
    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_mu} std={std_mu} support_fraction={support_fraction}"
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['n']}, mu={results[0]['mu']}\" fi
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
 'n=5, mu=4'}
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.2, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.2, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.2, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.1, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.5, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.2, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.1, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.5, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.4, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.1, 'instances_tested': 10, 'conjectur
TRIAL: {'metric_name': 'submodular_measure', 'metric_value': 3.3, 'instances_tested': 10, 'conjectur

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5bfd6fd7.py", line 106, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['n']}, mu={results[0]['mu']}\" first_fa
                                                ~~~~~~~~~~^^^^^
KeyError: 'n'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with KeyError: 'n', preventing reliable evaluation of counterexamples | next: Debug the test code's KeyError handling for 'n' key access

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	114794
2	propose	ollama_remote	qwen3:8b	0	0	84643
3	preregistration	ollama_remote	qwen3:8b	0	0	27504
4	novelty	ollama_remote	qwen3:8b	0	0	22811
5	novelty	ollama_remote	qwen3:8b	0	0	23236
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16596
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11531
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14056
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45281
10	judge	ollama_remote	qwen3:8b	0	0	121780

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 482232 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/c4e28a8fc7d7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c4e28a8fc7d7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c4e28a8fc7d7.tar.gz` (if generated)